

Java OOP Concepts

User-Defined Programs with User Input (Scanner)

#	Topic
1	Encapsulation
2	Object and Class
3	Method Overloading
4	Constructor, Static Variable, Method and Block
5	The 'this' Keyword
6	Inheritance
7	Method Overriding
8	The 'super' Keyword
9	The 'final' Keyword
10	Runtime Polymorphism
11	Abstract Class
12	Interface

Department of Computer Science

1. Encapsulation

Encapsulation wraps data (fields) and methods together in one class and restricts direct access using access modifiers (private, public, etc.).

Key Concepts:

- Data hiding using private fields
- Public getter/setter methods to access or modify private data
- Increases security and maintainability

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Encapsulation Example: BankAccount with User Input
public class BankAccount {
    private String accountHolder;
    private double balance;

    public BankAccount(String name, double initialBalance) {
        accountHolder = name;
        balance = initialBalance;
    }
    public String getAccountHolder() { return accountHolder; }
    public double getBalance() { return balance; }

    public void deposit(double amount) {
        if (amount > 0) { balance += amount; System.out.println("Deposited: Rs." + amount); }
        else System.out.println("Invalid deposit amount.");
    }
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) { balance -= amount; System.out.println("Withdrawn: Rs." + amount); }
        else System.out.println("Insufficient balance or invalid amount.");
    }
    public void displayInfo() {
        System.out.println("Account Holder : " + accountHolder);
        System.out.println("Balance : Rs." + balance);
    }
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter account holder name : ");
    String name = sc.nextLine();

    System.out.print("Enter initial balance (Rs.) : ");
    double initBalance = sc.nextDouble();

    BankAccount acc = new BankAccount(name, initBalance);
    acc.displayInfo();

    System.out.print("Enter deposit amount (Rs.) : ");
    double dep = sc.nextDouble();
    acc.deposit(dep);

    System.out.print("Enter withdraw amount (Rs.) : ");
    double wit = sc.nextDouble();
    acc.withdraw(wit);

    System.out.println("\n--- Updated Account Info ---");
}
```

```
        acc.displayInfo();
        sc.close();
    }
}

/* Sample Output:
Enter account holder name   : Alice
Enter initial balance (Rs.) : 5000
Account Holder : Alice | Balance : Rs.5000.0
Enter deposit amount  (Rs.) : 1500
Deposited: Rs.1500.0
Enter withdraw amount (Rs.) : 2000
Withdrawn: Rs.2000.0
Updated Balance : Rs.4500.0
*/
```

2. Object and Class

A Class is a blueprint; an Object is an instance of that class.

Key Concepts:

- Class defines structure and behavior
- Objects are created using the new keyword
- Multiple objects can be created from one class

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Object and Class Example: Student with User Input
public class Student {
    String name;
    int rollNumber;
    double marks;

    void displayInfo() {
        System.out.println("Name      : " + name);
        System.out.println("Roll Number : " + rollNumber);
        System.out.println("Marks      : " + marks);
        System.out.println("-----");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("How many students to enter? : ");
        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        Student[] students = new Student[n];

        for (int i = 0; i < n; i++) {
            students[i] = new Student();
            System.out.println("\n--- Enter details for Student " + (i+1) + " ---");
            System.out.print("Name      : "); students[i].name = sc.nextLine();
            System.out.print("Roll Number : "); students[i].rollNumber = sc.nextInt();
            System.out.print("Marks      : "); students[i].marks = sc.nextDouble();
            sc.nextLine();
        }

        System.out.println("\n==== Student Records =====");
        for (Student s : students) s.displayInfo();
        sc.close();
    }
}

/* Sample Output:
How many students to enter? : 2
--- Enter details for Student 1 ---
Name: Rahul | Roll: 101 | Marks: 89.5
--- Enter details for Student 2 ---
Name: Priya | Roll: 102 | Marks: 94.0
*/
```

3. Method Overloading

Method Overloading: same method name, different parameter lists.

It is a compile-time (static) polymorphism.

Key Concepts:

- Same method name, different number/type of parameters
- Resolved at compile time
- Return type alone cannot distinguish overloaded methods

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Method Overloading Example: Calculator with User Input
public class Calculator {

    int    add(int a, int b)           { return a + b; }
    int    add(int a, int b, int c)    { return a + b + c; }
    double add(double a, double b)     { return a + b; }
    String add(String s1, String s2)   { return s1 + s2; }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Calculator calc = new Calculator();

        // Integer addition
        System.out.print("Enter two integers (a b)           : ");
        int a = sc.nextInt(), b = sc.nextInt();
        System.out.println("add(" + a + ", " + b + ") = " + calc.add(a, b));

        // Three-integer addition
        System.out.print("Enter three integers (a b c)       : ");
        int x = sc.nextInt(), y = sc.nextInt(), z = sc.nextInt();
        System.out.println("add("+x+", "+y+", "+z+") = " + calc.add(x, y, z));

        // Double addition
        System.out.print("Enter two decimal numbers (a b)   : ");
        double d1 = sc.nextDouble(), d2 = sc.nextDouble();
        System.out.println("add(" + d1 + ", " + d2 + ") = " + calc.add(d1, d2));

        // String concatenation
        sc.nextLine();
        System.out.print("Enter first string : ");
        String s1 = sc.nextLine();
        System.out.print("Enter second string : ");
        String s2 = sc.nextLine();
        System.out.println("add(\"" + s1 + "\", \"" + s2 + "\") = " + calc.add(s1, s2));

        sc.close();
    }
}

/* Sample Output:
Enter two integers: 10 20 => add(10,20) = 30
Enter three integers: 10 20 30 => add(10,20,30) = 60
Enter decimals: 5.5 3.2 => add(5.5,3.2) = 8.7
Enter strings: Hello World => add = HelloWorld
*/
```

4. Constructor, Static Variable, Method and Block

Constructor : Called automatically when an object is created.

Static Variable: Shared across all objects of a class.

Static Method : Can be called without creating an object.

Static Block : Runs once when the class is first loaded into memory.

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Constructor, Static Variable, Method and Block - User Input Example
public class Student {

    static int totalStudents = 0;    // static variable

    // Static block - executes once when class loads
    static {
        System.out.println("[Static Block] Student class initialized.");
    }

    int studentId;
    String name;
    int age;

    // Constructor
    public Student(String name, int age) {
        totalStudents++;
        this.studentId = totalStudents;
        this.name = name;
        this.age = age;
        System.out.println("[Constructor] Student '" + name + "' created (ID=" + studentId + ")");
    }

    public static void showTotal() {    // static method
        System.out.println("Total students enrolled: " + totalStudents);
    }

    public void displayInfo() {
        System.out.println("ID: " + studentId + " | Name: " + name + " | Age: " + age);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        Student.showTotal();    // before creating any object

        System.out.print("\nHow many students to add? : ");
        int n = sc.nextInt();
        sc.nextLine();

        Student[] list = new Student[n];
        for (int i = 0; i < n; i++) {
            System.out.println("\n--- Student " + (i+1) + " ---");
            System.out.print("Enter name : "); String nm = sc.nextLine();
            System.out.print("Enter age : "); int ag = sc.nextInt(); sc.nextLine();
            list[i] = new Student(nm, ag);
        }

        System.out.println("\n===== Enrolled Students =====");
        for (Student s : list) s.displayInfo();
        Student.showTotal();
    }
}
```

```
        sc.close();
    }
}

/* Sample Output:
[Static Block] Student class initialized.
Total students enrolled: 0
How many students to add? : 2
[Constructor] Student 'Rahul' created (ID=1)
[Constructor] Student 'Priya' created (ID=2)
Total students enrolled: 2
*/
```

5. The 'this' Keyword

The this keyword refers to the current object.

Uses:

- Resolve naming conflict between fields and parameters
- Call another constructor of the same class (constructor chaining)
- Pass the current object as an argument to a method

Program (with User Input via Scanner):

```
import java.util.Scanner;

// 'this' Keyword Example: Employee with User Input
public class Employee {
    int id;
    String name;
    double salary;

    // 'this' resolves naming ambiguity between field and parameter
    public Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }

    // Constructor chaining using this()
    public Employee(int id, String name) {
        this(id, name, 25000.0); // default salary
        System.out.println(" [Default salary Rs.25000 applied via constructor chaining]");
    }

    public void printInfo() {
        System.out.println("ID: " + this.id + " | Name: " + this.name +
            " | Salary: Rs." + this.salary);
    }

    public void compareWith(Employee other) {
        if (this.salary > other.salary)
            System.out.println(this.name + " earns more than " + other.name);
        else if (this.salary < other.salary)
            System.out.println(other.name + " earns more than " + this.name);
        else
            System.out.println("Both earn the same salary.");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("--- Employee 1 (with custom salary) ---");
        System.out.print("Enter ID : "); int id1 = sc.nextInt(); sc.nextLine();
        System.out.print("Enter Name : "); String n1 = sc.nextLine();
        System.out.print("Enter Salary : "); double s1 = sc.nextDouble(); sc.nextLine();

        System.out.println("\n--- Employee 2 (default salary = Rs.25000) ---");
        System.out.print("Enter ID : "); int id2 = sc.nextInt(); sc.nextLine();
        System.out.print("Enter Name : "); String n2 = sc.nextLine();

        Employee e1 = new Employee(id1, n1, s1);
        Employee e2 = new Employee(id2, n2); // uses constructor chaining

        System.out.println("\n===== Employee Info =====");
```



```
        e1.printInfo();
        e2.printInfo();
        System.out.println("\n--- Comparison ---");
        e1.compareWith(e2);
        sc.close();
    }
}

/* Sample Output:
Employee 1: ID=1 | Name=Amit | Salary=Rs.55000
Employee 2: [Default salary Rs.25000 applied]
ID=2 | Name=Sara | Salary=Rs.25000
Amit earns more than Sara.
*/
```

6. Inheritance

Inheritance lets a child class acquire properties and methods of a parent class.

Types demonstrated:

- Single : Car extends Vehicle
- Multilevel : ElectricCar extends Car extends Vehicle

Note: Java does NOT support multiple inheritance through classes.

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Inheritance Example: Vehicle -> Car -> ElectricCar (User Input)
class Vehicle {
    String brand; int speed;
    public Vehicle(String brand, int speed) { this.brand=brand; this.speed=speed; }
    public void start() { System.out.println(brand + " is starting..."); }
    public void displayInfo() {
        System.out.println("Brand: " + brand + " | Top Speed: " + speed + " km/h");
    }
}

class Car extends Vehicle {
    int doors;
    public Car(String brand, int speed, int doors) {
        super(brand, speed); this.doors = doors;
    }
    public void displayCarInfo() {
        displayInfo();
        System.out.println("Number of Doors: " + doors);
    }
}

class ElectricCar extends Car {
    int batteryCapacity;
    public ElectricCar(String brand, int speed, int doors, int battery) {
        super(brand, speed, doors); this.batteryCapacity = battery;
    }
    public void displayElectricInfo() {
        displayCarInfo();
        System.out.println("Battery Capacity: " + batteryCapacity + " kWh");
    }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== Enter Electric Car Details ===");
        System.out.print("Brand name      : "); String brand = sc.nextLine();
        System.out.print("Top speed (km/h) : "); int speed = sc.nextInt();
        System.out.print("Number of doors  : "); int doors = sc.nextInt();
        System.out.print("Battery (kWh)   : "); int battery = sc.nextInt();

        ElectricCar ev = new ElectricCar(brand, speed, doors, battery);
        System.out.println("\n===== Electric Car Info =====");
        ev.start();
        ev.displayElectricInfo();
    }
}
```

```
        sc.close();
    }
}

/* Sample Output:
  Brand: Tesla | Top Speed: 250 km/h
  Number of Doors: 4
  Battery Capacity: 100 kWh
*/
```

7. Method Overriding

Method Overriding: subclass provides its own version of a parent method.

It is runtime (dynamic) polymorphism.

Rules:

- Same method name, same parameters, same return type
- Access must not be more restrictive than parent
- Use `@Override` annotation for safety

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Method Overriding Example: Shape hierarchy with User Input
class Shape {
    String color;
    public Shape(String color) { this.color = color; }
    public double area() { return 0; }
    public void displayInfo() {
        System.out.printf("%-10s | Color: %-8s | Area: %.2f%n",
                           getClass().getSimpleName(), color, area());
    }
}

class Circle extends Shape {
    double radius;
    public Circle(String color, double r) { super(color); radius = r; }
    @Override public double area() { return Math.PI * radius * radius; }
}

class Rectangle extends Shape {
    double length, width;
    public Rectangle(String color, double l, double w) { super(color); length=l; width=w; }
    @Override public double area() { return length * width; }
}

class Triangle extends Shape {
    double base, height;
    public Triangle(String color, double b, double h) { super(color); base=b; height=h; }
    @Override public double area() { return 0.5 * base * height; }
}

public class OverridingDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("--- Circle ---");
        System.out.print("Color : "); String c1 = sc.next();
        System.out.print("Radius : "); double r = sc.nextDouble();

        System.out.println("--- Rectangle ---");
        System.out.print("Color : "); String c2 = sc.next();
        System.out.print("Length : "); double l = sc.nextDouble();
        System.out.print("Width : "); double w = sc.nextDouble();

        System.out.println("--- Triangle ---");
        System.out.print("Color : "); String c3 = sc.next();
        System.out.print("Base : "); double b = sc.nextDouble();
        System.out.print("Height : "); double h = sc.nextDouble();
    }
}
```

```
Shape[] shapes = { new Circle(c1,r), new Rectangle(c2,l,w), new Triangle(c3,b,h) };

System.out.println("\nShape      | Color      | Area");
System.out.println("-----");
for (Shape s : shapes) s.displayInfo();
sc.close();
}
}

/* Sample Output:
Circle      | Color: Red      | Area: 153.94
Rectangle   | Color: Blue     | Area: 50.00
Triangle    | Color: Green    | Area: 24.00
*/
```

8. The 'super' Keyword

The super keyword refers to the immediate parent class.

Uses:

- Access parent fields or methods hidden by child class
- Call parent class methods overridden in child
- Invoke parent class constructor (must be first statement)

Program (with User Input via Scanner):

```
import java.util.Scanner;

// 'super' Keyword Example: Animal -> Dog (User Input)
class Animal {
    String name, sound;
    public Animal(String name, String sound) { this.name=name; this.sound=sound; }
    public void makeSound() { System.out.println(name + " says: " + sound); }
    public void displayInfo() { System.out.println("Animal: " + name); }
}

class Dog extends Animal {
    String breed;
    public Dog(String name, String breed) {
        super(name, "Woof"); // calls Animal constructor
        this.breed = breed;
    }
    @Override
    public void makeSound() {
        super.makeSound(); // calls Animal's makeSound()
        System.out.println(" (Dog-specific: " + name + " barks loudly!)");
    }
    @Override
    public void displayInfo() {
        super.displayInfo(); // calls Animal's displayInfo()
        System.out.println("Breed : " + breed);
    }
}

public class SuperDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("=== Enter Dog Details ===");
        System.out.print("Dog's name : ");
        String name = sc.nextLine();
        System.out.print("Dog's breed : ");
        String breed = sc.nextLine();

        Dog dog = new Dog(name, breed);

        System.out.println("\n--- Dog Information ---");
        dog.displayInfo();
        System.out.println("\n--- Sound Demo (super keyword) ---");
        dog.makeSound();
        sc.close();
    }
}

/* Sample Output:
Dog's name : Bruno
Dog's breed : Labrador
```

Animal: Bruno
Breed : Labrador
Bruno says: Woof
(Dog-specific: Bruno barks loudly!)

9. The 'final' Keyword

The final keyword prevents modifications:

- final variable : value cannot be changed (constant)
- final method : cannot be overridden in subclasses
- final class : cannot be extended (subclassed)

Program (with User Input via Scanner):

```
import java.util.Scanner;

// 'final' Keyword Example with User Input

final class TaxCalculator {           // final class - cannot be subclassed
    final double TAX_RATE = 18.0;    // final variable - constant

    public final double calculateTax(double income) { // final method
        return income * TAX_RATE / 100;
    }
    public final double netIncome(double income) {
        return income - calculateTax(income);
    }
}

class Circle {
    final double PI = 3.14159265;    // final variable
    double radius;
    public Circle(double r) { radius = r; }
    public final double area()        { return PI * radius * radius; }
    public final double circumference(){ return 2 * PI * radius; }
}

public class FinalDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Using final class TaxCalculator
        System.out.print("Enter your annual income (Rs.) : ");
        double income = sc.nextDouble();

        TaxCalculator tc = new TaxCalculator();
        System.out.printf("Tax Rate   : %.0f%%\n",      tc.TAX_RATE);
        System.out.printf("Tax Amount : Rs. %.2f%\n",   tc.calculateTax(income));
        System.out.printf("Net Income : Rs. %.2f%\n",   tc.netIncome(income));

        System.out.println();

        // Using final method in Circle
        System.out.print("Enter circle radius : ");
        double r = sc.nextDouble();

        Circle c = new Circle(r);
        System.out.printf("Area       : %.2f%\n",      c.area());
        System.out.printf("Circumference : %.2f%\n",   c.circumference());
        System.out.printf("(PI = %.5f - final variable)\n", c.PI);

        sc.close();
    }
}

/* Sample Output:
```


Enter annual income: 500000
Tax Rate : 18%
Tax Amount : Rs. 90000.00
Net Income : Rs. 410000.00
Enter radius: 7
Area: 153.94 | Circumference: 43.98

*/

10. Runtime Polymorphism

Runtime Polymorphism (Dynamic Method Dispatch): parent reference holds a child object; the method called is resolved at runtime by the JVM.

Key Concepts:

- Achieved through Method Overriding
- Parent reference -> Child object
- JVM decides which method to call at runtime
- Enables flexible and extensible design

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Runtime Polymorphism Example: Payment system with User Input
class Payment {
    String payerName;
    public Payment(String n) { payerName = n; }
    public void pay(double amount) {
        System.out.println(payerName + " is paying Rs." + amount);
    }
}

class CreditCardPayment extends Payment {
    public CreditCardPayment(String n) { super(n); }
    @Override public void pay(double a) {
        System.out.println(payerName + " paid Rs." + a + " via Credit Card.");
    }
}

class UPIPayment extends Payment {
    public UPIPayment(String n) { super(n); }
    @Override public void pay(double a) {
        System.out.println(payerName + " paid Rs." + a + " via UPI.");
    }
}

class NetBankingPayment extends Payment {
    public NetBankingPayment(String n) { super(n); }
    @Override public void pay(double a) {
        System.out.println(payerName + " paid Rs." + a + " via Net Banking.");
    }
}

public class PolymorphismDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter your name   : ");
        String name = sc.nextLine();
        System.out.print("Enter amount (Rs.): ");
        double amount = sc.nextDouble();

        System.out.println("\nChoose payment method:");
        System.out.println(" 1. Credit Card");
        System.out.println(" 2. UPI");
        System.out.println(" 3. Net Banking");
        System.out.print("Enter choice (1/2/3): ");
    }
}
```

```
int choice = sc.nextInt();

// Parent reference holds child object -> Runtime Polymorphism
Payment p;
switch (choice) {
    case 1: p = new CreditCardPayment(name); break;
    case 2: p = new UPIPayment(name); break;
    case 3: p = new NetBankingPayment(name); break;
    default: System.out.println("Invalid choice!"); return;
}
System.out.println();
p.pay(amount); // method resolved at runtime
sc.close();
}

/* Sample Output:
Name: Alice | Amount: 2500
Choice: 1 (Credit Card)
Alice paid Rs.2500.0 via Credit Card.
*/
```

11. Abstract Class

An Abstract Class cannot be instantiated and may declare abstract methods (without a body). Subclasses must implement all abstract methods.

Key Concepts:

- Declared with the abstract keyword
- Can have both abstract and concrete methods
- Cannot create objects of an abstract class directly
- Defines a common template for related subclasses

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Abstract Class Example: Employee Payroll with User Input
abstract class Employee {
    String name; int id;
    public Employee(String name, int id) { this.name=name; this.id=id; }
    public abstract double calculateSalary();
    public void displayInfo() {
        System.out.printf("ID: %-4d | Name: %-15s | Salary: Rs. %.2f%n",
            id, name, calculateSalary());
    }
}

class FullTimeEmployee extends Employee {
    double monthlySalary;
    public FullTimeEmployee(String n, int id, double s) { super(n,id); monthlySalary=s; }
    @Override public double calculateSalary() { return monthlySalary; }
}

class PartTimeEmployee extends Employee {
    int hours; double rate;
    public PartTimeEmployee(String n, int id, int h, double r) { super(n,id); hours=h; rate=r; }
    @Override public double calculateSalary() { return hours * rate; }
}

class ContractEmployee extends Employee {
    double fee, taxRate;
    public ContractEmployee(String n, int id, double f, double t) { super(n,id); fee=f; taxRate=t; }
    @Override public double calculateSalary() { return fee - (fee * taxRate / 100); }
}

public class AbstractDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Full-time employee
        System.out.println("=== Full-Time Employee ===");
        System.out.print("Name          : "); String n1 = sc.nextLine();
        System.out.print("ID            : "); int id1 = sc.nextInt();
        System.out.print("Monthly Salary : "); double sal = sc.nextDouble(); sc.nextLine();

        // Part-time employee
        System.out.println("\n=== Part-Time Employee ===");
        System.out.print("Name          : "); String n2 = sc.nextLine();
        System.out.print("ID            : "); int id2 = sc.nextInt();
        System.out.print("Hours Worked  : "); int hrs = sc.nextInt();
    }
}
```

```

        System.out.print("Hourly Rate      : "); double rate = sc.nextDouble(); sc.nextLine();

        // Contract employee
        System.out.println("\n=== Contract Employee ===");
        System.out.print("Name          : "); String n3 = sc.nextLine();
        System.out.print("ID            : "); int id3 = sc.nextInt();
        System.out.print("Project Fee    : "); double fee = sc.nextDouble();
        System.out.print("Tax Rate (%)   : "); double tax = sc.nextDouble();

        Employee[] list = {
            new FullTimeEmployee(n1, id1, sal),
            new PartTimeEmployee(n2, id2, hrs, rate),
            new ContractEmployee(n3, id3, fee, tax)
        };

        System.out.println("\n===== Payroll Summary =====");
        for (Employee e : list) e.displayInfo();
        sc.close();
    }
}

/* Sample Output:
    FullTime   : ID=201 | Neha   | Salary: Rs. 45000.00
    PartTime   : ID=202 | Raj    | Salary: Rs. 12000.00
    Contract   : ID=203 | Suman  | Salary: Rs. 54000.00
*/

```

12. Interface

An Interface contains only abstract methods and constants by default.

It achieves 100% abstraction and supports multiple inheritance.

Key Concepts:

- Methods are public and abstract by default
- Fields are public, static, and final by default
- A class uses the implements keyword
- A class can implement multiple interfaces

Program (with User Input via Scanner):

```
import java.util.Scanner;

// Interface Example: Smart Home Devices with User Input
interface Switchable {
    void turnOn();
    void turnOff();
}

interface WiFiConnectable {
    void connectToWiFi(String network);
    void disconnectFromWiFi();
}

class SmartLight implements Switchable {
    String location;
    public SmartLight(String loc) { location = loc; }
    @Override public void turnOn() { System.out.println("Light at '"+location+"' is ON."); }
    @Override public void turnOff() { System.out.println("Light at '"+location+"' is OFF."); }
}

class SmartTV implements Switchable, WiFiConnectable {
    String brand;
    public SmartTV(String brand) { this.brand = brand; }
    @Override public void turnOn() { System.out.println(brand+" TV is ON."); }
    @Override public void turnOff() { System.out.println(brand+" TV is OFF."); }
    @Override public void connectToWiFi(String net) {
        System.out.println(brand+" TV connected to: " + net); }
    @Override public void disconnectFromWiFi() {
        System.out.println(brand+" TV disconnected from WiFi."); }
}

public class InterfaceDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Smart Light location : ");
        String location = sc.nextLine();
        SmartLight light = new SmartLight(location);
        light.turnOn();
        light.turnOff();

        System.out.println();
        System.out.print("Enter Smart TV brand : ");
        String brand = sc.nextLine();
        System.out.print("Enter WiFi network name : ");
        String wifi = sc.nextLine();
    }
}
```

```
        SmartTV tv = new SmartTV(brand);
        tv.turnOn();
        tv.connectToWiFi(wifi);
        tv.disconnectFromWiFi();
        tv.turnOff();
        sc.close();
    }
}

/* Sample Output:
    Smart Light location : Living Room
    Light at 'Living Room' is ON.
    Light at 'Living Room' is OFF.
    Smart TV brand: Samsung | WiFi: Home_5G
    Samsung TV is ON.
    Samsung TV connected to: Home_5G
    Samsung TV disconnected from WiFi.
    Samsung TV is OFF.
*/
```
